

HOW DID YOU MANAGE YOUR PROJECT?

EMEE20002 Management Assessment

Contents

- Project Summary and Charter
- Team Structure
- Planning
- Executing
- Outcomes
- Lessons Learned

PROJECT SUMMARY AND CHARTER

EEME20002

Project Summary

- Who we are
- Functional Description
 - Task overview
 - Receive command, process and return result
 - Data Processor
 - Find the information of peak
 - Command Processor
 - NNN detect
 - Control TX, RX and DP

Project Charter: Overall

Project Charter		Date	
		28/11/2025	
Title FPGA Peak Detector		Team ID Group 8	Project Manager Xinrui Zhu
Goals	Roles	Responsibilities	
Design VHDL programmes to interpret user's command and detect the peak value and six adjacent ones in a sequence up to 500 samples, then implement it on FPGA by the end of TB2.	Xinrui Zhu: • Leader of command processor (CP) Yuan Ren • CP implementer Julia Yang: • CP implementer Xi Zhao • Leader of data processor (DP) Yuhao Jiang: • DP implementer	Xinrui Zhu: • CP architecture and ports define Yuan Ren: • CP implementation on VHDL Julia Yang: • Integrate with FPGA Tx/Rx Xi Zhao: • Design DP algorithm Yuhao Jiang: • DP implementation on VHDL	
In-Scope	Out-of-Scope	Stakeholders	
Data processor RTL Command processor RTL FPGA synthesis Test and verification	Testbenches Data generator Serial communication of Tx/Rx on FPGA	All teammates <u>Dr.</u> Roshan <u>Dr.</u> Karamah Teaching assistants	

Project Charter: In Scope

- DP & CP development
 - From algorithm to HDL
 - From simulation to synthesis & on board
 - Testbench for internal small modules
- Integration with provided modules
- Documentation

Project Charter: Out of Scope

- Data generator
- TX, RX
- Testbenches
 - Only for some internal small submodules
- Power, area
- Performance
 - Only latency considered

Project Charter: Constraints

- Specifications
 - 100MHz at minimum
 - Hardware
 - Specified FPGA model
 - Limited computation resources
 - Compatibility with provided modules
- Deadline
 - 16th, April

Project Charter: Deliverables & Targets

- Deliverables
 - VHDL source code
 - Technical report
 - Video on group management
- Targets
 - Learn digital design process
 - Learn group collaboration

TEAM STRUCTURE

Team Structure: Overall

- Data Processor Team
 - Yuan Ren
 - Julia Yang
- Command Processor Team
 - Xinrui Zhu
 - Yuhao Jiang
 - Xi Zhao

Team Structure: Yuan Ren

- Lead
 - VHDL implementation
 - Code review
 - Report documentation
- Support
 - Simulation and debugging
 - Meeting organisation

Team Structure: Julia Yang

- Lead

- Algorithm & FSM design
- Simulation and debugging
- Report Writing (DP part)

- Support

- System integration
- Liaison with teaching staff

Team Structure: Xinrui Zhu

- Lead
 - System architecture design
 - Core FSM/ASM design
 - System integration
 - GitHub repository management
- Support
 - VHDL implementation
 - Code review

Team Structure: Yuhao Jiang

- Lead
 - IO controller design
 - VHDL implementation
- Support
 - Simulation and debugging
 - Progress tracking
 - Report documentation (CP part)

Team Structure: Xi Zhao

- Lead
 - Simulation and debugging
 - Report Writing (CP part)
 - Progress tracking
- Support
 - VHDL implementation
 - Code review

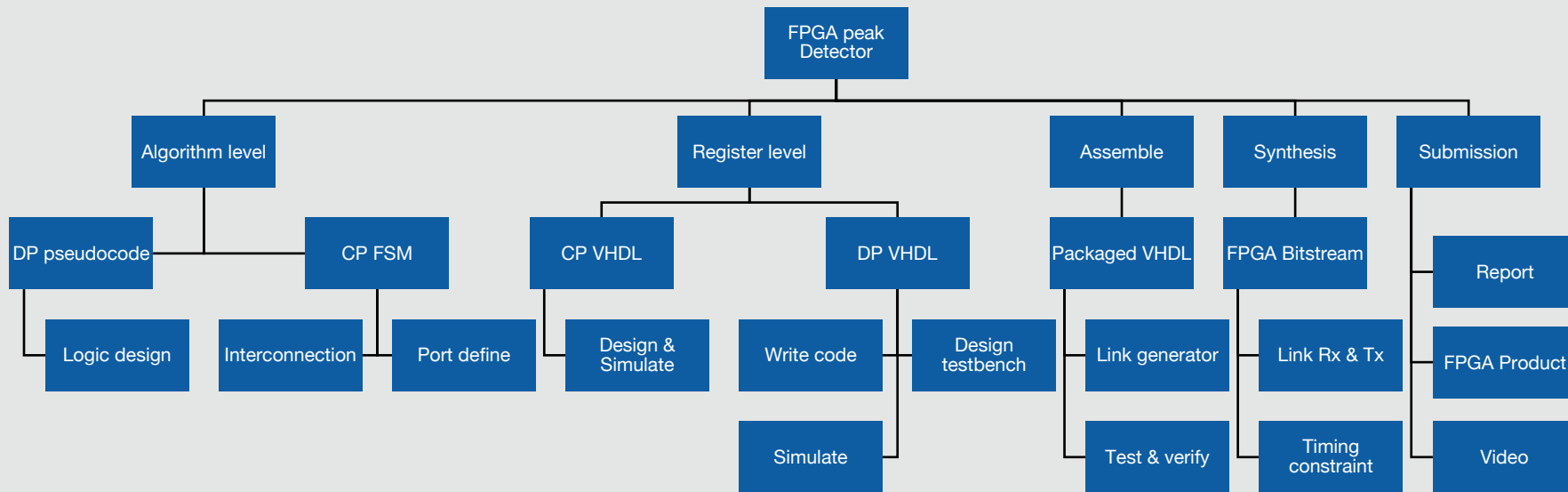
PLANNING

EEME20002

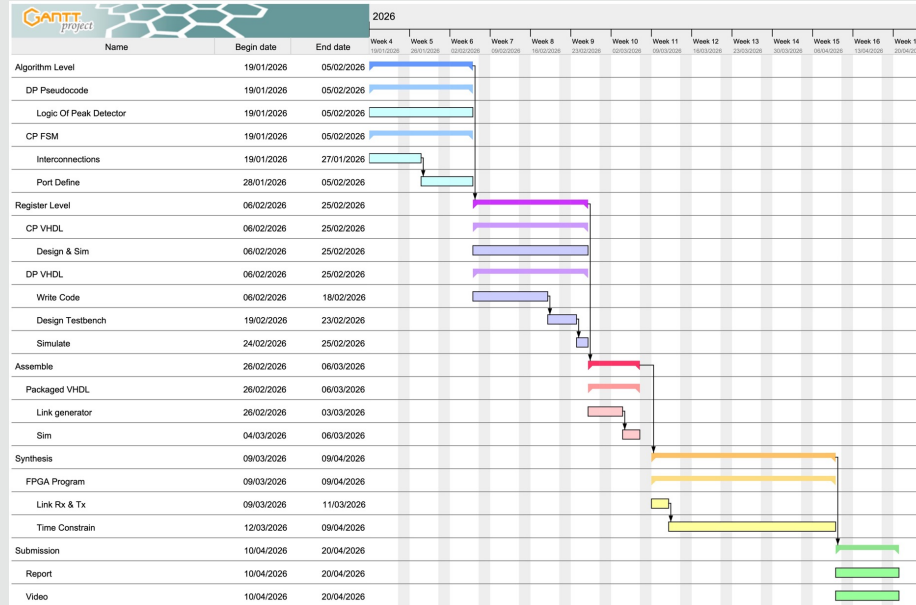
Planning: Methodology

- Task division
 - Work breakdown structure (WBS)
- Timeline & stage deliverables
 - Gantt chart
- Risk control
 - Risk register table

Planning: Work Breakdown Structure



Planning: Gantt Chart



Planning: Risk Register

Date	Raised By	Description	Likelihood	Severity	Risk	Mitigation	Owner
28/11/2025	Xi Zhao	Member doesn't complete his own work before a certain deadline.	2	2	4	Arrange weekly meeting to check individual's progress.	Xi Zhao
28/11/2025	Xinrui Zhu, Yuan Ren	Simulation result doesn't meet expectation. (e.g. can't detect peak)	4	4	16	Draft the code after the algorithm is finished, pay attention to syntax errors.	Xinrui Zhu
28/11/2025	Xinrui Zhu	FPGA timing doesn't converge	4	5	20	Get familiar with FPGA workflow from lectures.	Xinrui Zhu
28/11/2025	Yuan Ren	Team member becomes unavailable (e.g. accidentally injured)	1	5	5	Optimise work distribution and prepare for online collaboration.	Yuan Ren
29/11/2025	Yuhao Jiang	FPGA board is malfunctioned or damaged	1	5	5	Test the functions at first time, contact university staff for replacement early.	Yuhao Jiang
01/12/2025	Julia Yang	Inconsistent coding styles of developers leads to inconsistent module interfaces	3	1	3	Write ReadMe document to define essential specs, add comments next to critical code sections.	Julia Yang

Planning: Limitations

- Limited task information available
- Limited familiarity with FPGA development process

EXECUTING

EEME20002

Executing: Coordination

- Meetings in person
 - Two three-hour sessions
 - Focus on project development
- Online meetings on Teams
 - Learn, evaluate and prepare for next-step
- GitHub: code collaboration
 - GoofyAsh6795395/FPGA-peak-detector

February 2026				
Monday	Tuesday	Wednesday	Thursday	Friday
02	03	04	05	06
10:00 Electrical and Electrn	14:00 Engineering Mather	10:00 Electrical and Electrn	12:00 Electromagnetics ar	9:00 Electrical and Electroi
13:00 Engineering Mather	20:00 Command Processo	11:00 Y2 EENG Tutorial	13:00 Engineering Mather	13:00 Electromagnetics ar
14:00 Electromagnetics ar		13:50 SBBO: 1:1 Student S	15:00 Electrical Energy Co	14:00 Engineering Mather
17:00 Engineering Mather		19:00 Data Processor Ever	17:00 Engineering Mather	
09	10	11	12	13
10:00 Electrical and Electrn	10:00 Electrical and Electrn	18:30 Data_Processor_Cor	12:00 Electromagnetics ar	9:00 Electrical and Electroi
13:00 Engineering Mather	14:00 Engineering Mather		13:00 Engineering Mather	13:00 Electromagnetics ar
14:00 Electromagnetics ar	18:30 Command Group Ex		15:00 Electrical Energy Co	14:00 Engineering Mather
17:00 Engineering Mather			17:00 Engineering Mather	
16	17	18	19	20
9:00 Online session of D.P.	10:00 Electrical and Electrn	10:00 Electrical and Electrn	12:00 Electromagnetics ar	9:00 Electrical and Electroi
10:00 Electrical and Electrn	14:00 Engineering Mather	11:00 Y2 EENG Tutorial	13:00 Engineering Mather	13:00 Electromagnetics ar
13:00 Engineering Mather	15:00 Electrical Energy Co	19:00 Never Knows Best	14:00 EENG Y2 Town Hall	14:00 Engineering Mather
+2			+2	
23	24	25	26	27
10:00 eep group work	10:00 eep group work		13:00 Engineering Mather	8:00 EMS
13:00 Engineering Mather			17:00 Engineering Mather	
17:00 Engineering Mather				

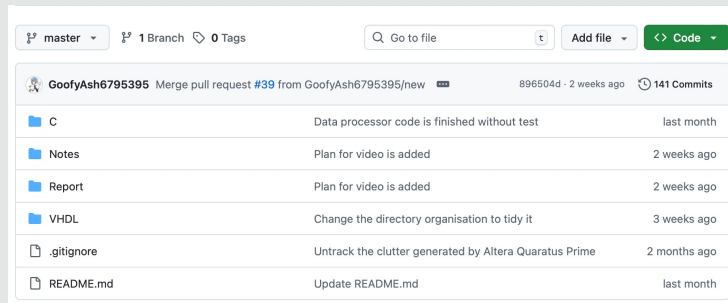
Executing: Coordination

- Meetings in person
 - Two three-hour sessions
 - Focus on project development
- Online meetings on Teams
 - Learn, evaluate and prepare for next-step
- GitHub: code collaboration
 - GoofyAsh6795395/FPGA-peak-detector

February 2026				
Monday	Tuesday	Wednesday	Thursday	Friday
02	03	04	05	06
10:00 Electrical and Electrn	14:00 Engineering Mather	10:00 Electrical and Electrn	12:00 Electromagnetics ar	9:00 Electrical and Electroi
13:00 Engineering Mather	20:00 Command Processo	11:00 Y2 EENG Tutorial	13:00 Engineering Mather	13:00 Electromagnetics ar
14:00 Electromagnetics ar		13:50 SBBO: 1:1 Student S	15:00 Electrical Energy Co	14:00 Engineering Mather
17:00 Engineering Mather		19:00 Data Processor Ever	17:00 Engineering Mather	
09	10	11	12	13
10:00 Electrical and Electrn	10:00 Electrical and Electrn	18:30 Data_Processor_Cor	12:00 Electromagnetics ar	9:00 Electrical and Electroi
13:00 Engineering Mather	14:00 Engineering Mather		13:00 Engineering Mather	13:00 Electromagnetics ar
14:00 Electromagnetics ar	18:30 Command Group Ex		15:00 Electrical Energy Co	14:00 Engineering Mather
17:00 Engineering Mather			17:00 Engineering Mather	
16	17	18	19	20
9:00 Online session of D.P.	10:00 Electrical and Electrn	10:00 Electrical and Electrn	12:00 Electromagnetics ar	9:00 Electrical and Electroi
10:00 Electrical and Electrn	14:00 Engineering Mather	11:00 Y2 EENG Tutorial	13:00 Engineering Mather	13:00 Electromagnetics ar
13:00 Engineering Mather	15:00 Electrical Energy Co	19:00 Never Knows Best	14:00 EENG Y2 Town Hall	14:00 Engineering Mather
+2			+2	
23	24	25	26	27
10:00 eeep group work↻	10:00 eeep group work↻		13:00 Engineering Mather	8:00 EMS
13:00 Engineering Mather			17:00 Engineering Mather	
17:00 Engineering Mather				

Executing: Coordination

- Meetings in person
 - Two three-hour sessions
 - Focus on project development
- Online meetings on Teams
 - Learn, evaluate and prepare for next-step
- GitHub: code collaboration
 - GoofyAsh6795395/FPGA-peak-detector



Executing: Progress Tracking

- Report orally in person
- Communicate on IM software
- On GitHub
 - Blame
 - Pull requests
 - Commit and File changes
- In-code development log
 - Rationale, outstanding tasks

Older			Newer			Contributors 1	
2 months ago		Note updated, with more practi...	76	<code>begin</code>			
last month		Correct the indentation added...	77	<code>numWords <= NNN_reg;</code>			
last month		Delete unnecessary codes for ...	78	--The output of numWords is directly connected to the internal register of *NNN_reg			
			79	--Not use the input from parser directly in case of glitches.			
			80	--No need to worry about the type of this output because there is another type-conv			
last month		Correct the indentation added...	81				
2 months ago		Add patches on parser	82	<code>state_transition_logic:</code>			
3 weeks ago		Tidy the code, interchange th...	83	<code>process(current_state, isL, isP, isANN, finished, counterL, counterP, counterData,</code>			
2 months ago		Scheduler is completed witho...	84	<code>begin</code>			
			85	<code>next_state <= current_state;</code>			
			86	<code>case current_state is</code>			
			87	<code>when idle =></code>			
3 weeks ago		The feature of delimiter printin...	88	<code>if isANN = '1' or isP = '1' or isL = '1' then</code>			
3 weeks ago		Tidy the code, interchange th...	89	<code>next_state <= printSep;</code>			
2 months ago		Scheduler is completed witho...	90	<code>end if;</code>			
			91	<code>when run =></code>			
			92	<code>if dataReady = '1' then</code>			
			93	<code>next_state <= printData;</code>			
			94	<code>else</code>			
			95	<code>next_state <= run;</code>			
			96	<code>end if;</code>			
			97	<code>when printData =></code>			
			98	<code>--State state transition if counter condition is met.</code>			
3 weeks ago		Correct the start logic, remov...	99	<code>if counterData = 2 and print_ack = '1' then</code>			
last month		Modify some files based on fe...	100	<code>--This means that two data have already been sent previously, and t</code>			
last month		Correct the indentation added...	101	<code>--So state can be updated to next one now.</code>			
last month		A nearly refreshed design so l...	102	<code>--If there isn't anything awaiting print, even it should not exist.</code>			

Executing: Progress Tracking

- Report orally in person
- Communicate on IM software
- On GitHub
 - Blame
 - Pull requests
 - Commit and File changes
- In-code development log
 - Rationale, outstanding tasks

```
134 134      when idle =>
135 135          if start = '1' then
136 136              +      --Update before actual manipulation happens, thus, the boundary should be correspondingly changes.
137 137              +      --Likewise, ++i, rather than i++;
138 138          if finished = '1' then
139 139              --This means that, the last cycle is end and now a new cycle start.
140 140              finished_reg <= '0';
141 141          --Flush the stored data
142 142          counter_next <= 0;
143 143          counter_next <= 1;
144 144          max_reg <= 0;
145 145          max_index_reg <= 0;
146 146          result_next <= (others => (others => '0'));
147 147      @@ -149,11 +151,10 @@ begin
148 148          tens := to_integer(unsigned(numWords_bcd(1)));
149 149          ones := to_integer(unsigned(numWords_bcd(0)));
150 150          NNN_reg <= (hundreds * 100 + tens * 10 + ones);
151 151      else
152 152          counter_next <= counter + 1;
153 153          --Update before actual manipulation happens, thus, the boundary should be correspondingly changes.
154 154          --Likewise, ++i, rather than i++;
155 155      end if;
156 156      counter_next <= counter + 1;
157 157      --Must be placed here instead above.
158 158  end if;
```

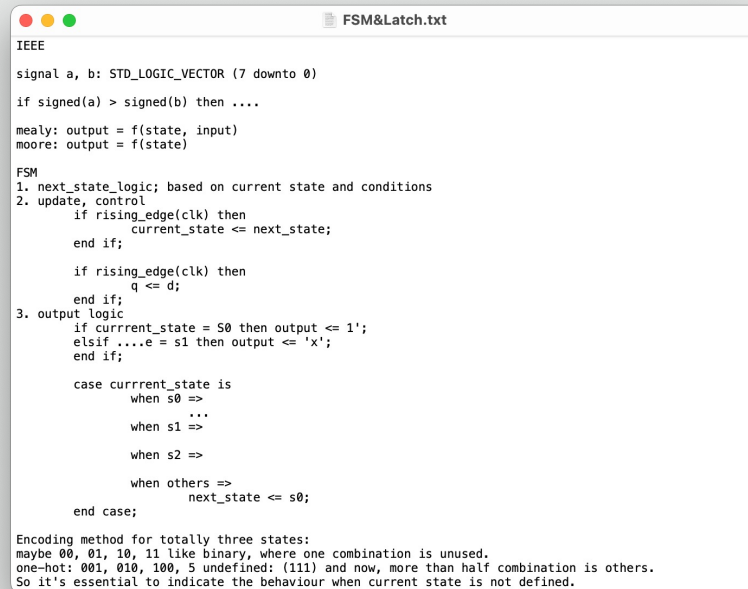
Executing: Progress Tracking

- Report orally in person
- Communicate on IM software
- On GitHub
 - Blame
 - Pull requests
 - Commit and File changes
- In-code development log
 - Rationale, outstanding tasks

```
568 --Modified at 2am, 10/03/2026:
569 -- Now this code could successfully deal with a unstable upstream source.
570 -- A flag "mistake" is used to capture if another dataReady is received while printing.
571 -- This flag is clean when printing has been executed for totally 3 time, and the flag is still high level.
572 -- Another signal, data_mistake, is also declared, to capture the byte signal at a wrong time.
573 -- Both of them are managed in clocking process.
574 -- The function is verified by simulation, as expected.
575 -- However, seems that the simulation only involve a normal ANNN and corresponding task.
576 -- More complex operations, like L command, P command, echo interrupt are scheduled in future.
577 -- To test the behaviour under such operations in real hardware.
578 --
579 --Modified on 16/03/2026:
580 -- The problem of termination is identified and it's found to be caused by the incomplete state transition logic.
581 -- Now corrected and give another trial.
582 --
583 --New feature at 9am, 18/03/2026:
584 -- A new state is introduced for the purpose of printing CRLF.
585 --
586 --Modification on 19/03/06:
587 -- Adjust the start logic so that it's now a strict single cycle pulse
588 -- And the bonus is, the dataConsume from university will not repeatedly retrieve new values;
589 -- So, another modification is, removing the whole mistake logic, there should not.
590 -- The problem of LSB slice of L command is spotted from board test and the boundary is corrected.
591 --
592 --Modification at 9am, at 20/03/2026:
593 -- Try to add some probes for command L to identify the pattern and then deduce what causes the problem.
594 -- Finally, the upper and lower logic is found wrong.
595 -- Corrected, and change the endian to match the data processor signals.
596 --
597 --New function at 3pm, at 20/03/2026:
598 -- Now it can fully print the delimiter.
599 --
600 --Tidy at 11pm, at 20/03/2026:
601 -- I have write another separated clocking process,
602 -- And now, one is for the register of the statistics of this system.
603 -- What's contained in the another one are, registered ports.
```

Executing: Technical Issues

- Limited familiarity with GitHub
 - Centralised management
- Uneven retention of VHDL
- Limited prior exposure to Vivado
 - Targeted online tutorials



```
IEEE
signal a, b: STD_LOGIC_VECTOR (7 downto 0)
if signed(a) > signed(b) then ...
mealy: output = f(state, input)
moore: output = f(state)

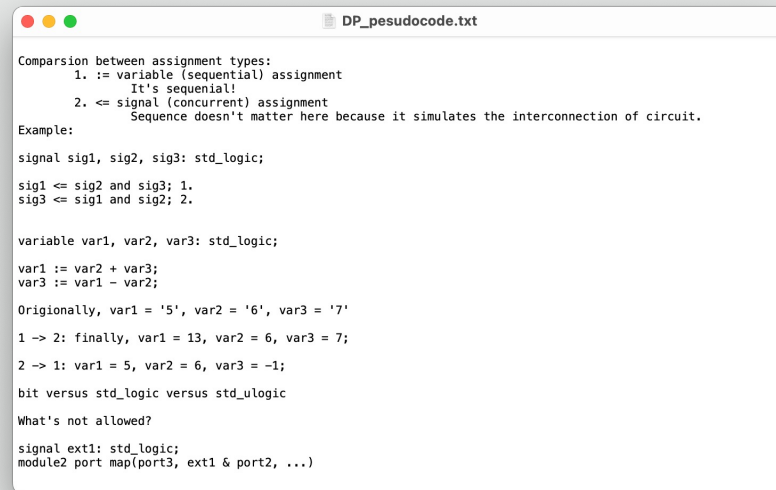
FSM
1. next_state_logic; based on current state and conditions
2. update, control
   if rising_edge(clk) then
     current_state <= next_state;
   end if;
   if rising_edge(clk) then
     q <= d;
   end if;
3. output logic
   if current_state = S0 then output <= '1';
   elsif ....e = s1 then output <= 'x';
   end if;

   case currrent_state is
     when s0 =>
       ..
     when s1 =>
       ..
     when s2 =>
       ..
     when others =>
       next_state <= s0;
   end case;

Encoding method for totally three states:
maybe 00, 01, 10, 11 like binary, where one combination is unused.
one-hot: 001, 010, 100, 5 undefined: (111) and now, more than half combination is others.
So it's essential to indicate the behaviour when current state is not defined.
```

Executing: Technical Issues

- Limited familiarity with GitHub
 - Centralised management
- Uneven retention of VHDL
- Limited prior exposure to Vivado
 - Targeted online tutorials



```
DP_pesudocode.txt

Comparsion between assignment types:
1. := variable (sequential) assignment
   It's sequential!
2. <= signal (concurrent) assignment
   Sequence doesn't matter here because it simulates the interconnection of circuit.

Example:

signal sig1, sig2, sig3: std_logic;

sig1 <= sig2 and sig3; 1.
sig3 <= sig1 and sig2; 2.

variable var1, var2, var3: std_logic;

var1 := var2 + var3;
var3 := var1 - var2;

Origionally, var1 = '5', var2 = '6', var3 = '7'
1 -> 2: finally, var1 = 13, var2 = 6, var3 = 7;
2 -> 1: var1 = 5, var2 = 6, var3 = -1;

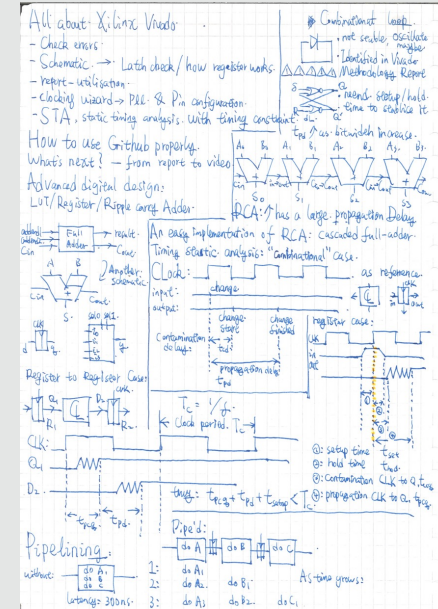
bit versus std_logic versus std_uologic

What's not allowed?

signal ext1: std_logic;
module2 port map(port3, ext1 & port2, ...)
```

Executing: Technical Issues

- Limited familiarity with GitHub
 - Centralised management
- Uneven retention of VHDL
- Limited prior exposure to Vivado
 - Targeted online tutorials



Executing: Management Issues

- Inaccurate time estimation
 - Longer than expectation
 - Algorithm & Simulation
 - Short than expectation
 - Assemble
- Incomplete task division
 - Video & Report part
- Re-evaluate the task
 - Based on progress and difficulty
- Leave time slack for some deliverables
- Update WBS, Gantt Chart

OUTCOMES

EEME20002

Outcomes: FPGA Product

- Functional requirements satisfied
- Enhanced echo function
 - Not interrupted by output stream
- Higher frequency than expected
 - CP: 183 MHz
 - DP: 175 MHz
 - Overall: 151 MHz

Outcomes: Team collaboration

- Clear task allocation
- Regular meetings
- Internal knowledge sharing

LESSONS LEARNED

EEME20002

Lessons Learned: In Summary

- Project management tools
 - Gantt chart, WBS, risk register
- Collaboration tools
 - Teams, GitHub
- Technical
 - Digital design approach: VHDL syntax, FSM/ASM design
 - Industrial toolchain
 - Questa: VHDL writing, simulation
 - Xilinx Vivado Design Suite: synthesis, implementation

Lessons Learned: In Future

- Better task interpretation before planning
- A more detailed plan
 - Include non-technical tasks
- Improve communication efficiency
 - Prepare before meeting
 - Take actions after meeting